Chapter 3
ARM Assembly Language

3.1 ARM Assembly Language and programming environment

ARM assembly language

- ARM instructions are written in the form

```
Label  Op-code operand1, operand2, operand3  //comment
```

- Consider the following example implementing a loop.

```
Test_5 ADD  r0,r1,r2     //TotalTime = Time + NewTime
       SUBS r7,#1 //Decrement loop counter
       BNE  Test_5       //IF not zero (Z = 0)
                         //THEN goto Test_5
```

- The Label field is a user-defined label that can be used by other instructions to refer to that line.
- Any text following a double slash is regarded as a comment field and is ignored by the assembler.

ARM assembly language and CPUlator

- We use CPUlator Simulator as the programming environment in this course. More details about this simulator will be provided in Lab 1.
- Web-based simulator and no installation needed: https://cpulator.01xz.net/?sys=arm-de1soc
- Consider this program that you will see in Lab 1:

```
.org    0x1000  // Start at memory location 1000
.text           // Code section
.global _start
_start:
mov r0, #9       // Store decimal 9 in register r0
mov r1, #0xE     // Store hex E (decimal 14) in register r1
add r2, r1, r0  // Add the contents of r0 and r1 and put result in r2

// End program
_stop:
b   _stop

.end
```

ARM assembly language and CPUlator

- Two types of statements:
  1. Assembler directives
     - Used to control the assembler
     - Do not generate machine code
     - Highlighted in red.
     - All directives in CPUlator starts with a period.

  2. Executable instructions
     - Do generate machine code to perform various operations
     - *Pseudoinstruction*: instruction available to the programmer but not part of the processor's instruction set. Shorthand expression that an assembler converts to appropriate machine code (e.g., ldr r1, =0xFF896788 – discuss in more detail after covering the fundamentals).

Directives used in the sample program

- .org
  - Format: .org memory_address
  - tells the compiler where in memory to place the program.
- .text
  - identifies the code section of the program.
- .global
  - Format: .global label
  - identifies globally visible labels.
- .end
  - identifies the end of the source code and data.

Instructions used in the sample program

`mov` (Move)

- Puts a value into a register.
- Two formats:
  - `mov r0, #9` puts the (immediate) decimal value 9 into register r0.
  - `mov r1, r0` copies the value in register r0 into register r1.

`add` (Add)

- adds two values and places the results into a register.
- Two formats:
  - `add r2, r1, r0` adds the contents of r0 and r1 and puts the result in register r2.
  - `add r3, r1, #7` adds the contents of r1 and the decimal value 7 and puts the result in register r3.

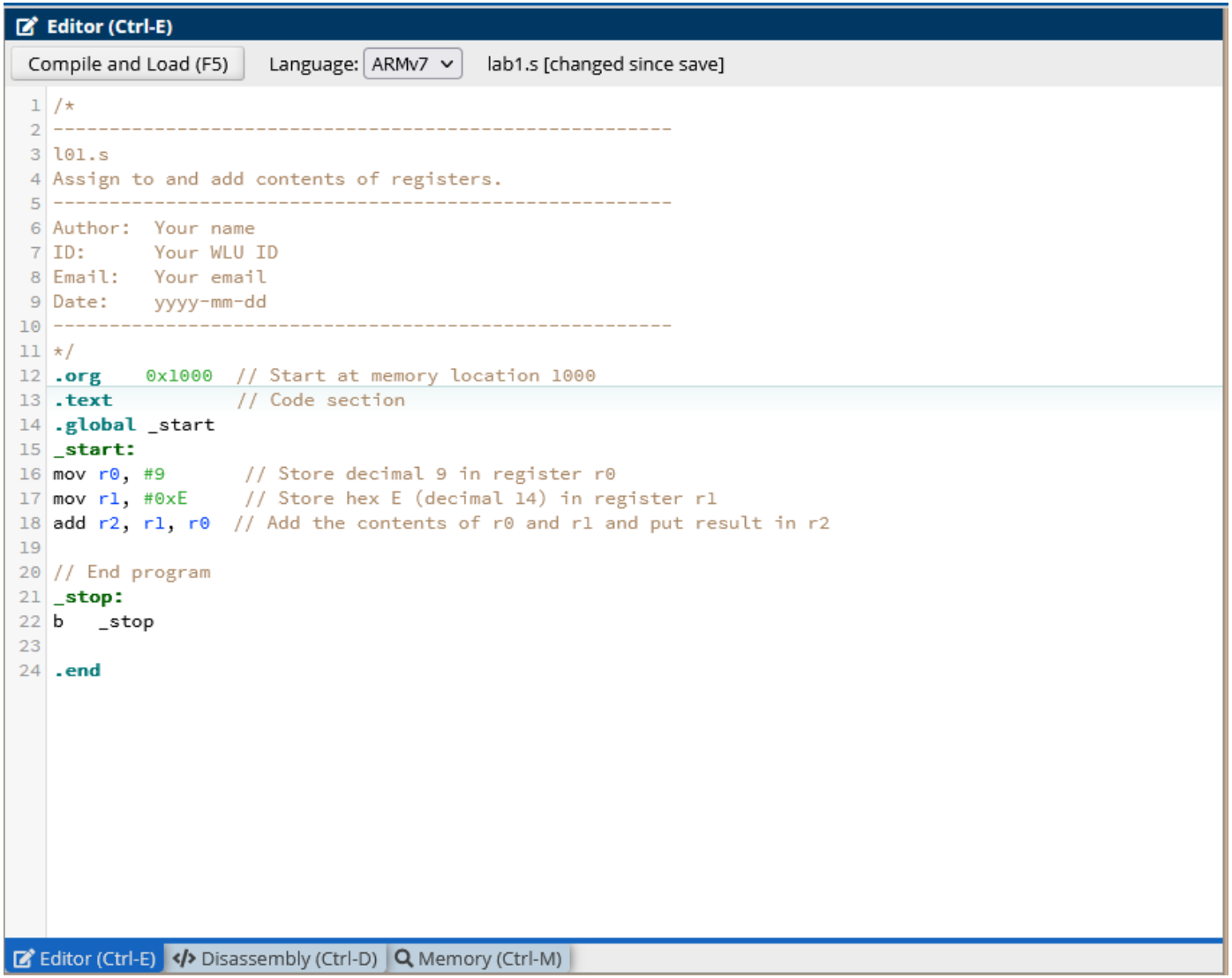
`b` (Branch)

- Branches execution to a memory location.
- The instruction:

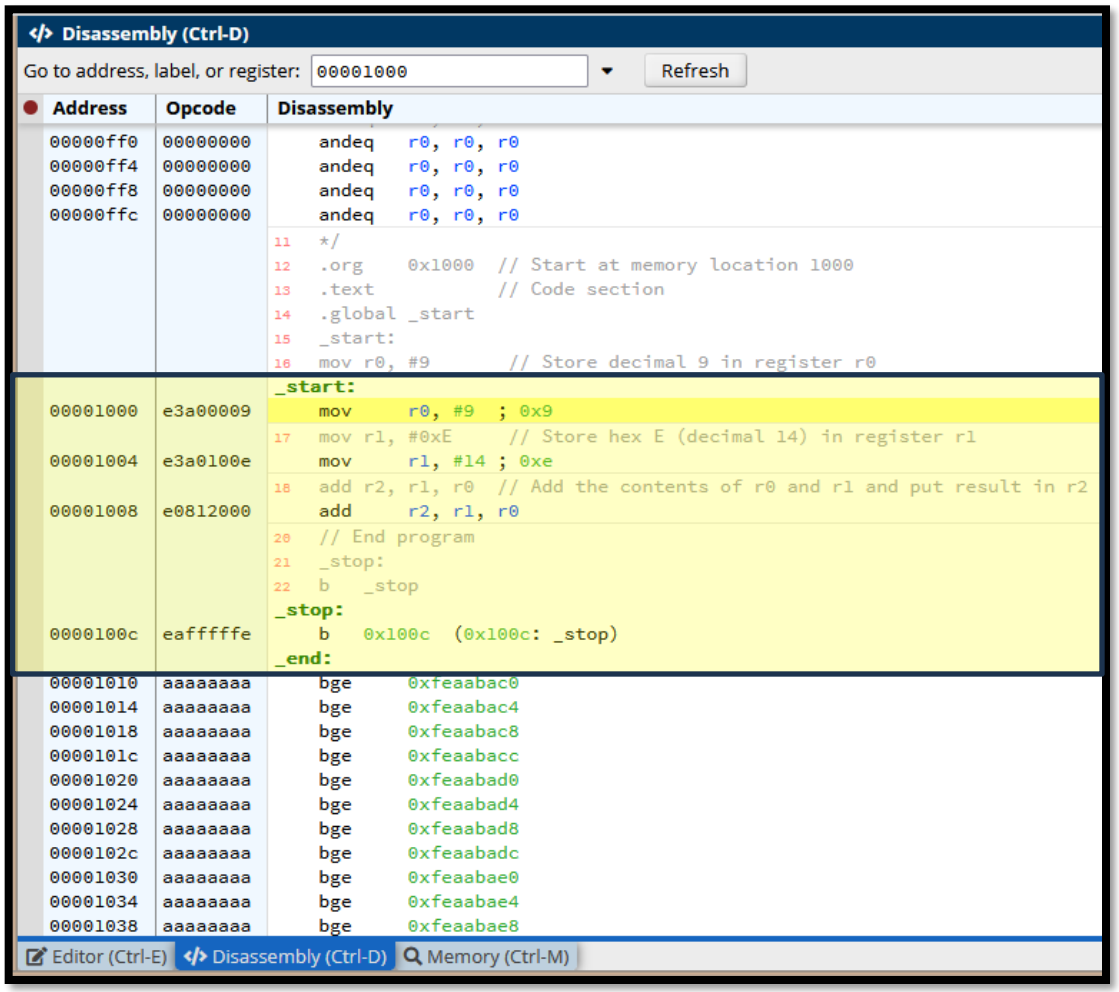
```
_stop:
b _stop
```

terminates a program by branching to its own location.

CPUlator interface



CPUlator interface



Address

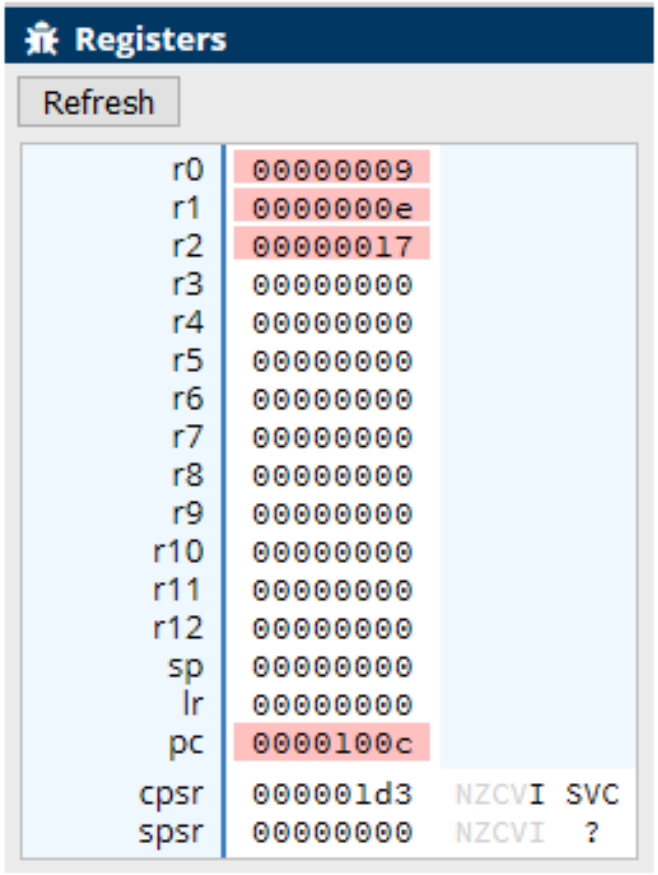
- The address of the instruction.
- The program starts at address 0x1000 as specified by the .org instruction.

Opcode

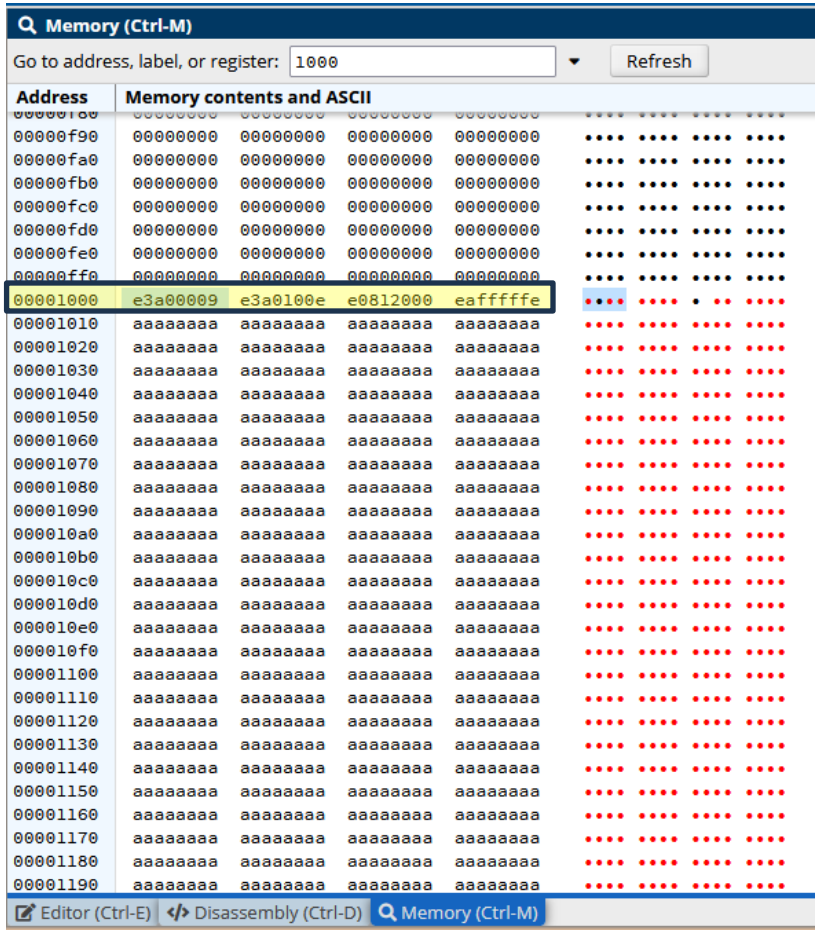
- 32-bit instruction encoded in binary format
- Specify the operation (e.g., ADD) and the operands.

CPUlator interface

Registers



Memory



CPUlator interface

- You will learn the use of debugging tools CPUlator in Lab 2 and 3.
- The following statement you encounter in Lab 2 is a pseudoinstruction:
  - `ldr r0, =0x12345678`
  - This pseudoinstruction puts a number into a register
  - You will learn more why we need this statement and how this pseudoinstruction works later.

3.2 ARM Data-processing instructions

ARM Data Processing Instructions

- Consist of :
  - Arithmetic: `ADD` `ADC` `SUB` `SBC` `RSB` `RSC`
  - Logical: `AND` `ORR` `EOR` `BIC`
  - Comparisons: `CMP` `CMN` `TST` `TEQ`
  - Data movement: `MOV` `MVN`
- These instructions only work on registers, NOT memory.
- Syntax:

  `<Operation>{<cond>}{S} Rd, Rn, Operand2`
  - Comparisons set flags only - they do not specify Rd
  - Data movement does not specify Rn

- Second operand is sent to the ALU via *barrel shifter*.

Updating ARM conditional codes

- ARM does not automatically update its status flags after an operation
- *Update-on-demand*: status flags are updated only the programmer puts an "S" suffix in the mnemonic, i.e.,
  - ADD r1, r2, r3 does not update status flags
  - ADDS r1, r2, r3 updates status flags
- This allows the programmer performs a test, and then carry out other instructions without changing the flags:

```
SUBS r1, r1, #1      ; subtract 1 from r1 and set status bits
ADD r2, r2, #4       ; increment r2 (don't update status bits)
BEQ Error            ; if r1 is zero then deal with the problem
```

🧠 Micro-challenges: Update-on-demand

- You execute the following instructions in order:

```
ADD   r0, r1, r2
SUBS  r3, r4, r5
```

- **Question:**
  Which instruction(s) update the CPSR condition flags?

A. Only ADD
B. Only SUBS
C. Both instructions
D. Neither instruction

🧠 Micro-challenges: Update-on-demand 1

- You execute the following instructions in order:

```
ADD   r0, r1, r2
SUBS  r3, r4, r5
```

- **Question:**
  Which instruction(s) update the CPSR condition flags?

A. Only ADD
B. Only SUBS
C. Both instructions
D. Neither instruction

🧠 Micro-challenges: Update-on-demand 2

Why does ARM use *update-on-demand* rather than automatically updating flags after every arithmetic instruction?

Choose **all** that apply.

A. Allows arithmetic without overwriting previous test results
B. Reduces instruction encoding size
C. Gives programmers fine control over flag updates
D. Prevents overflow detection

Addition

- Suppose you have a computer capable of adding 8-bit values but you want to add two 16-bit values.
- You can divide the 16-bit addition to two 8-bit additions, but the answer is not correct if there is a carry in the addition of the least significant byte:

Addition of two 8-bit number without carry

```
  34   32
+ 57   DF
-------------
  8B   11
```

Addition of two 8-bit number with carry

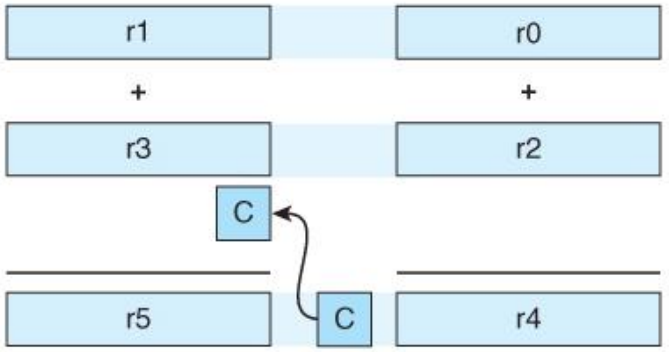
```
  34   32
+ 57   DF
-------------
  8C   11
```



Multi-Word Addition

- The following shows how a 64-bit addition is performed:

```
ADDS r4, r0, r2
ADC  r5, r1, r3
```



- Must use S after ADD, as the C flag in the status register (CPSR) must be updated.
- `ADC`: add with carry

Example

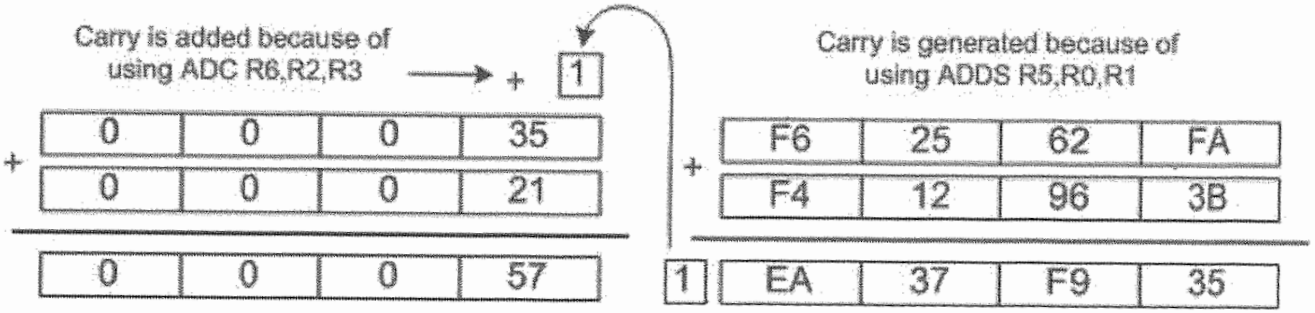
Analyze the following program which adds 0x35F62562FA to 0x21F412963B:

35F62562FA
+21F412963B

```
LDR    R0, =0xF62562FA    ; R0 = 0xF62562FA
LDR    R1, =0xF412963B    ; R1 = 0xF412963B
MOV    R2, #0x35          ; R2 = 0x35
MOV    R3, #0x21          ; R3 = 0x21
ADDS   R5, R1, R0         ; R5 = 0xF62562FA + 0xF412963B
                          ; now C = 1
ADC    R6, R2, R3         ; R6 = R2 + R3 + C
                          ;    = 0x35 + 21 + 1 = 0x57
```

Solution:

After the R5 = R0 + R1 the carry flag is one. Since C = 1, when ADC is executed, R6 = R2 + R3 + C = 0x35 + 0x21 + 1 = 0x57.



Microsoft Windows calculator support data size of up 64-bit (double word). Use it to verify the above calculations.

🧠 Micro-challenges: Multi-Word Addition

You are adding two **64-bit unsigned integers** stored as follows:
First operand: low = r0, high = r1
Second operand: low = r2, high = r3

The **64-bit result must be stored in r4 (low word) and r5 (high word)**.
Which instruction sequence correctly performs this operation?

A.
```
ADD   r4, r0, r2
ADC   r5, r1, r3
```

B.
```
ADDS  r4, r0, r2
ADC   r5, r1, r3
```

C.
```
ADD   r5, r1, r3
ADD   r4, r0, r2
```

D.
```
ADDS  r5, r1, r3
ADD   r4, r0, r2
```

Subtraction

Subtraction:

- `SUB r1, r2, r3`: `[r1]←[r2]-[r3]`
- `SBC`: subtract with borrow

Reverse subtraction:

- `RSB r1, r2, r3: [r1]←[r3]-[r2]`
- As the literal must be in the second operand, RSB allows you to subtract the register content from a literal, e.g.,
  - `RSB r1, r2, #10`: `[r1]←10-[r2]`
- No negation operation in ARM. RSB also allows you to implement negation:
  - `RSB r1, r1, #0: [r1]← 0-[r1]`

Examples: SUBS

Show the steps involved for the following cases:

a)

```
MOV   R2, #0x4F          ; R2 = 0x4F
MOV   R3, #0x39          ; R3 = 0x39
SUBS  R4, R2, R3         ; R4 = R2 - R3
```

b)

```
MOV   R2, #0x4F          ; R2 = 0x4F
SUBS  R4, R2, #0x05      ; R4 = R2 - 0x05
```

Solution:

a)

```
  0x4F        0000004F
- 0x39      + FFFFFFC7   2's complement of 0x39
  16        1 00000016
```

Note: if C = 1, No Borrow
if C = 0, Borrow

The flags would be set as follows: C = 1, Z = 0, N = 0 and V = 0

b)

```
  0x4F        0000004F
- 0x05      +  FFFFFFFB  2's complement of 0x05
  0x4A       1 0000004A
```

C = 1, Z = 0, N = 0 and V = 0

Example: Multi-Word Subtraction

Analyze the following program which subtracts 0x21F62562FA from 0x35F412963B:

35F412963B
- 21F62562FA

```
LDR    R0, =0xF62562FA    ; R0 = 0xF62562FA,
                          ; notice the syntax for LDR
LDR    R1, =0xF412963B    ; R1 = 0xF412963B
MOV    R2, #0x21          ; R2 = 0x21
MOV    R3, #0x35          ; R3 = 0x35
SUBS   R5, R1, R0         ; R5 = R1 - R0
                          ;    = 0xF412963B - 0xF62562FA, and C = 0
SBC    R6, R3, R2         ; R6 = R3 - R2 - 1 + C
                          ;    = 0x35 - 0x21 - 1 + 0 = 0x13
```

Solution:

After the R5 = R1 – R0 there is a borrow so the carry flag is cleared. Since C = 0, when SBC is executed,

R6 = R3 – R2 – 1 + C = 0x35 – 0x21 – 1 + 0 = 0x35 – 0x21 – 1= 0x13.

F412963B
- F62562FA

00000035
- 00000021

C = 0 so there is borrow

SBC R6,R3,R2 => R6 = C - 1 + R3 - R2
R6 = C -1+ R3 + (2's complement of R2)

SUBS R5,R1,R0 =>
R5 = R1 + (2's complement of R0)

	0	0	0	35
+	FF	FF	FF	DF
+ 0 - 1	0	0	0	13

	F4	12	96	3B
+	09	DA	9D	06
0	FD	ED	33	41

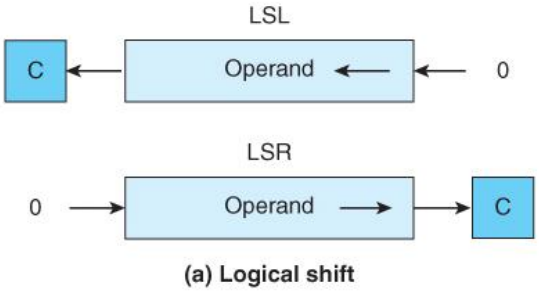
Bitwise logical operations

Logical instruction[25]	Operation	Final value in r2
`AND r2,r1,r0`	11001010.00001111	00001010
`OR r2,r1,r0`	11001010+00001111	11001111
`NOT r2, r1`	$\overline{11001010}$	00110101
`EOR r2,r1,r0`	11001010⊕00001111	11000101

- ARM does not have a NOT instruction of this form, but the NOT instruction can be implemented by one of the following two approaches:
  1. `mvn r2, r1`: mvn copies the logical complement of r1 into r2
  2. Apply EOR with the second operand equal to 0xFFFFFFFF. Note for each bit x, $x\ EOR\ 1\ = \bar{x}$

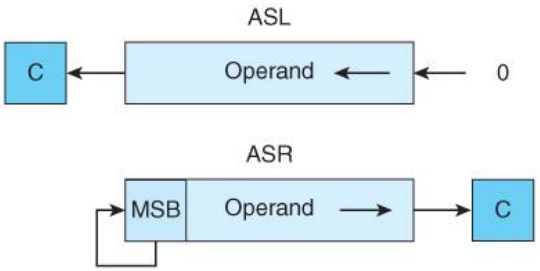
     (i.e., 0 EOR 1 = 1
     1 EOR 1 = 0)

Shift operations



(a) Logical shift

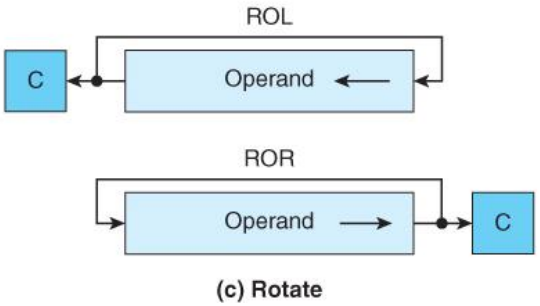
In a logical shift, a zero is shifted in and the bit shifted out is copied to the carry bit of the condition code register.



(b) Arithmetic shift

In an arithmetic shift, the number is either multiplied by 2 (ASL) or divided by 2 (ASR). The sign of a two's complement number is preserved.

The bit shifted out is copied into the carry bit.



(c) Rotate

In a rotate operation, the bit shifted out is copied into the bit vacated at the other end (i.e., no bit is lost during a rotate). The bit shifted out is also copied into the carry bit.

© Cengage Learning 2014

ASR:
(Divide by 2)

+ve: 00100000 → 00010000

-ve: 10000100 → 11000010

10000100

Magnitude?
01111100 = 7C

What is 10000100 ÷ 2?
Shift magnitude one bit to right
00111110 = 3E
2'comp:
11000010 (-3E)

The Barrel Shifter in ARM

LSL : Logical Left Shift



Multiplication by a power of 2

ASR: Arithmetic Right Shift



Division by a power of 2, preserving the sign bit

e.g.,

$10000100 = -7C_{16} = -124_{10}$

$11000010 = -3E_{16} = -62_{10}$

LSR : Logical Shift Right



Division by a power of 2

ROR: Rotate Right



Bit rotate with wrap around from LSB to MSB

Using the Barrel shifter: The second operand

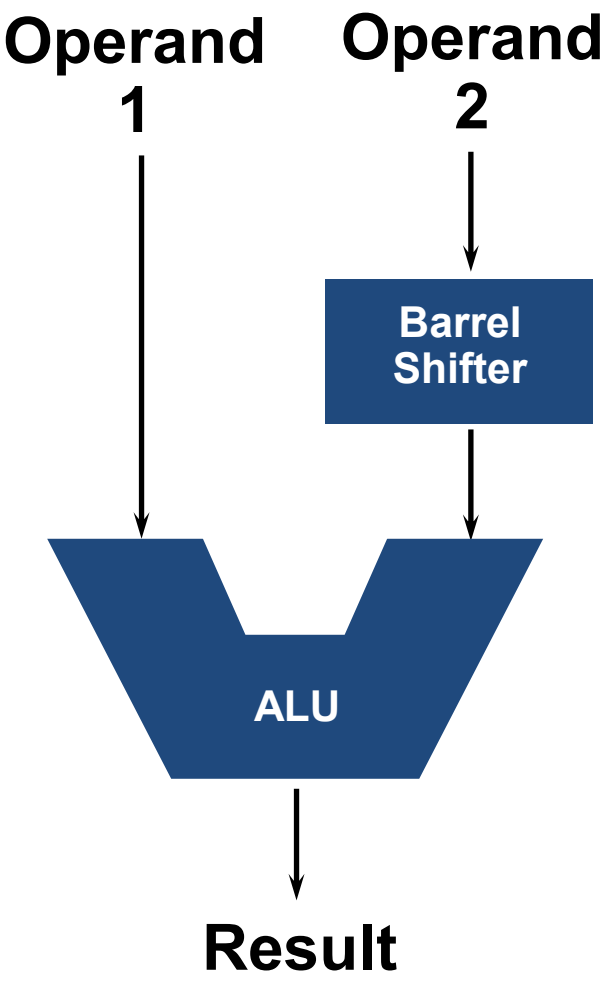
- ARM combines shifting with other data processing operations, because the second operand can be shifted before it is used. Consider:

```
ADD r0,r1,r2, LSL #1
```

- A logical shift left is applied to the contents of r2 before they are added to the contents of r1. This operation is equivalent to [r0] ← [r1] + [r2] x 2.

- Logical shift without other operations can be achieved by, e.g.:

```
MOV r0, r0, LSL #1
```



Static vs. Dynamic Shifts

Static shift

- The number of shifts is fixed in the code.
- e.g., `ADD r0,r1,r2, LSL #1` implements `[r0]←[r1]+[r2]x2`

Dynamic shift

- Consider `ADD r0,r1,r2, LSL r3`, which implements `[r0]←[r1]+[r2]x2`$^{[r3]}$
- This allows you to change the number of shifts at runtime.

🧠 Micro-challenge: Static Shift

Assume the following register contents **before execution**:

[r1] = 0x00000005

[r2] = 0x00000003

The instruction executed is:

```
ADD r0, r1, r2, LSL #2
```

What is the value stored in r0 after the instruction executes?

A. 0x00000008
B. 0x00000011
C. 0x00000014
D. 0x00000020

🧠 Micro-challenge: Dynamic Shift

Assume the following register contents **before execution**:
[r1] = 0x00000010
[r2] = 0x00000003
[r3] = 0x00000002

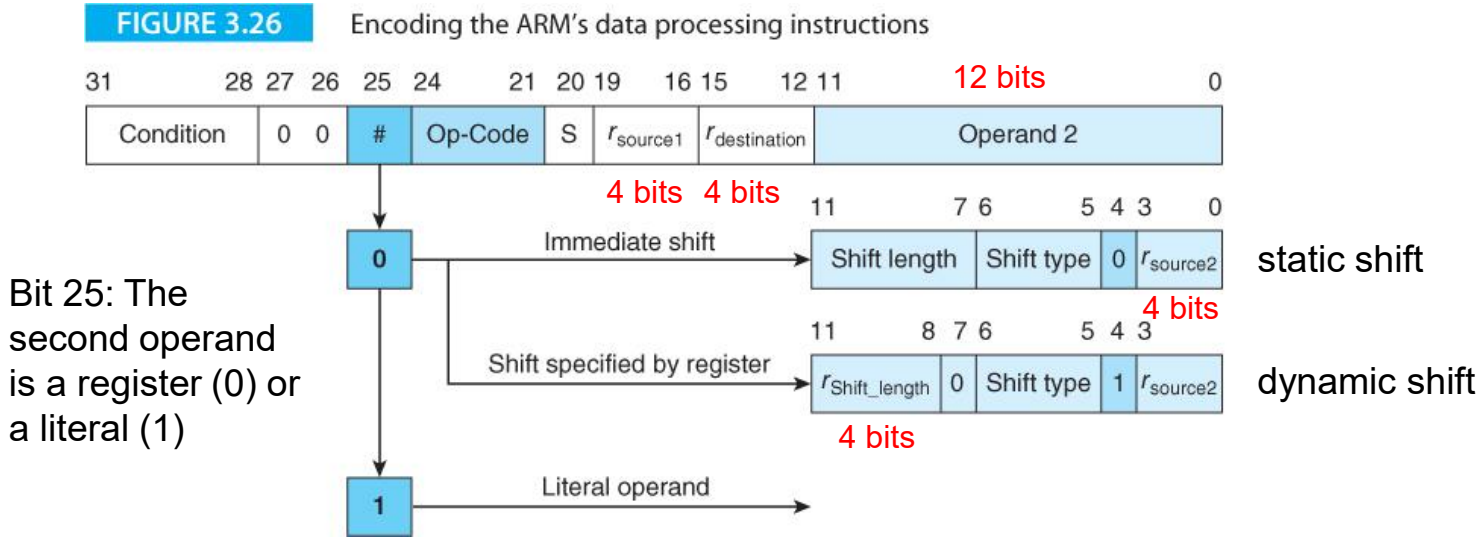
The instruction executed is:

```
ADD r0, r1, r2, LSL r3
```

Question:
What is the value stored in r0 after the instruction executes?

A. 0x00000016
B. 0x0000001C
C. 0x00000028
D. 0x00000040

Instruction Encoding



- 4 bits for each of the destination and first source operand.
- 12 bits for the second operand consisting of:
  - shift type
  - static shift: 5-bit unsigned integer in Bit 7 to 11 specifies the number of shifts. Range is 0 to 31.
  - dynamic shift: 4-bit address of the register in Bit 8 to 11 specifies the register containing the number of shifts.

🧠 Micro-challenge: Static Shift Range

Which of the following static shift specifications is **illegal** and will cause an assembler error?

A. MOV r0, r1, LSL #0
B. ADD r2, r3, r4, LSR #31
C. SUB r5, r6, r7, ASR #32
D. EOR r8, r9, r10, LSL #16

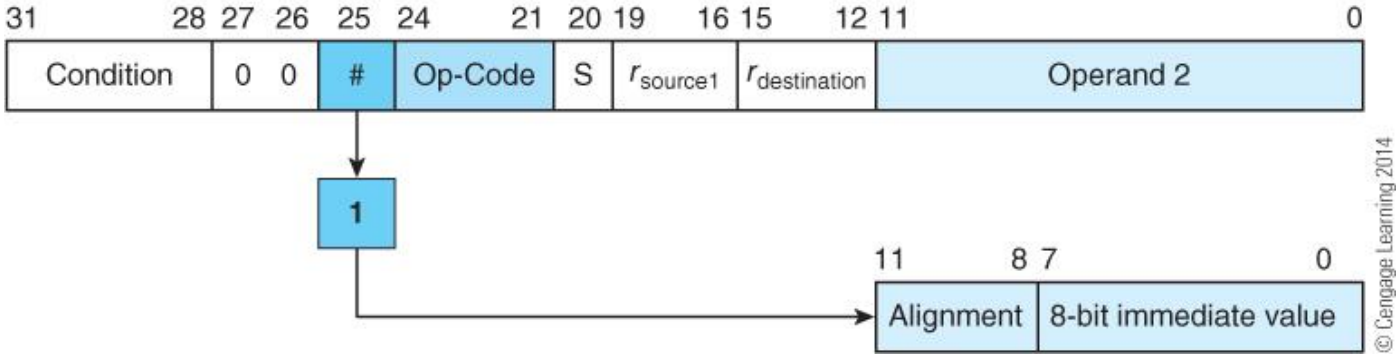
Instruction Encoding

- Bit 21-24: Opcode

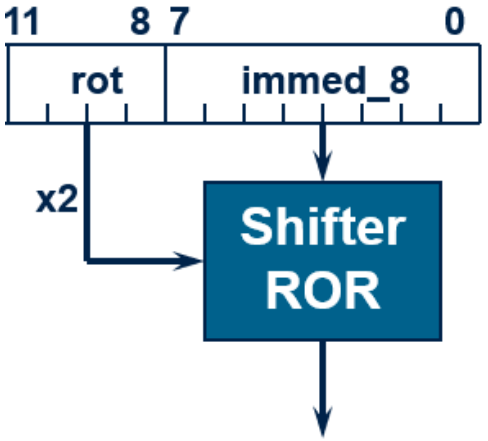
0000 = AND - Rd:= Op1 AND Op2
0001 = EOR - Rd:= Op1 EOR Op2
0010 = SUB - Rd:= Op1 - Op2
0011 = RSB - Rd:= Op2 - Op1
0100 = ADD - Rd:= Op1 + Op2
0101 = ADC - Rd:= Op1 + Op2 + C
0110 = SBC - Rd:= Op1 - Op2 + C - 1
0111 = RSC - Rd:= Op2 - Op1 + C - 1
1000 = TST - set condition codes on Op1 AND Op2
1001 = TEQ - set condition codes on Op1 EOR Op2
1010 = CMP - set condition codes on Op1 - Op2
1011 = CMN - set condition codes on Op1 + Op2
1100 = ORR - Rd:= Op1 OR Op2
1101 = MOV - Rd:= Op2
1110 = BIC - Rd:= Op1 AND NOT Op2
1111 = MVN - Rd:= NOT Op2

Literal Encoding

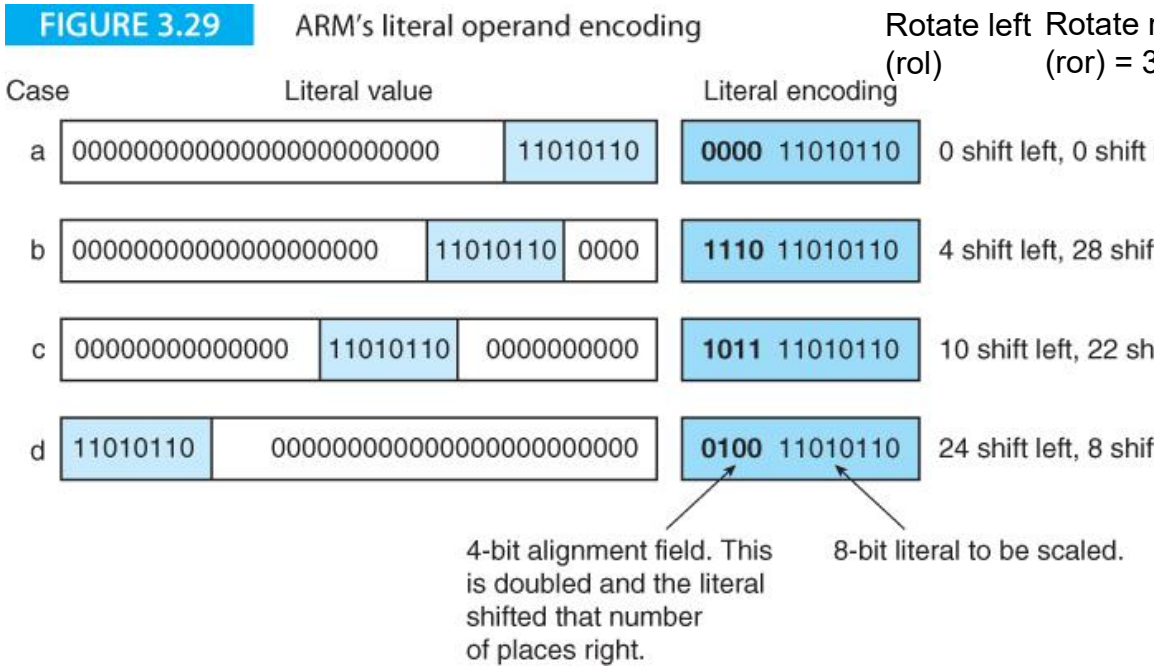
FIGURE 3.28 Diagram of ARM's literal operand encoding



- No ARM instruction can contain a 32-bit immediate constant
- The data processing instruction format has 12 bits available for operand2
- The literal processed by the instruction is the 8-bit value stored in Bits 0-7, shifted by ROR by 2 times the value stored in Bits 8-11.



Example of literal encoding



Rotate left (rol) | Rotate right (ror) = 32 - rol

Encode ror shift divided by 2

0 shift left, 0 shift right → Encode 0

4 shift left, 28 shift right → 28 = 2 x14
Encode 14 decimal or E in hex

10 shift left, 22 shift right → 22 = 2 x11
Encode 11 decimal or B in hex

24 shift left, 8 shift right → 8 = 2 x 4
Encode 4 decimal or 4 in hex

Example of literal encoding

- The assembler converts immediate values to the rotate form:
  - `MOV r0,#4096; uses 0x01, left shift 12, ror 20, encode 10 or A`

  `machine code: E3A00A01`
  - `ADD r1,r2,#0xFF0000; uses 0xFF, left shift 16, ror 16, encode 8`

  `machine code: E2821 8FF`
- Should be able to convert least significant 12 bits of machine code to the literal being encoded
- The MVN (move negative) instruction MVN r0, #XX can specify unshifted constant in the range 0xFFFFFF00 to 0XFFFFFFFF. e.g.,
  - **`MVN r0, #0xFF`** moves 0xFFFFFF00 to r0
- Values that cannot be generated in this way will cause an error. e.g.,
  - **`add r0, r1, #0x00004567`** cannot be encoded as a shifted 8-bit number.

🧠 Micro-challenge: Literal encoding 1

Which of the following **can be encoded directly** as an immediate operand in an ARM data-processing instruction? Choose **all** that can be encoded.

A. #0x000000FF
B. #0x00004567
C. #0xFF000000
D. #0x55555555

🧠 Micro-challenge: Literal encoding 2

The following machine code is given:

`E3A01C02`

Which literal value is encoded in this instruction?

A. 0x00000002
B. 0x00000200
C. 0x00002000
D. 0x02000000

🧠 Micro-challenge: Literal encoding 3

Determine the literal operand encoded by the following 32-bit machine code. Express your answer in hexadecimal representation.

(a) E3A00C67

(b) E2810E89

(c) E283260F

(d) E3A0091E

🧠 Micro-challenge: Literal encoding 3

Determine the literal operand encoded by the following 32-bit machine code. Express your answer in 32-bit hexadecimal representation.

(a) E3A00C67
Encoded C → ror $12 \times 2 = 24$ → left shift = 32-24 = 8.
Literal = 67 shifted by 8 bits to the left = 00006700

(b) E2810E89
Encoded E → ror $14 \times 2 = 28$ → left shift = 32-28 = 4.
Literal = 89 shifted by 4 bits to the left = 00000890

(c) E28326 0F
Encoded 6 → ror $6 \times 2 = 12$ → left shift = 32-12 = 20.
Literal = 0F shifted by 20 bits to the left = 00F00000

🧠 Micro-challenge: Literal encoding 3

The number of bits to shift does not have to be a multiple of 4:

(d) E3A00**9**1E

Encoded 9 → ror $9 \times 2 = 18$ → left shift = 32-18 = 14.

Literal = 1E shifted by 14 bits to the left:

Shift 1E in binary:

00 01 11 10 00 00 00 00 00 00 00 00

= 78000

- In 32-bits: 00078000

Multiplication

- Take two m-bit operands → output a 2m-bit result.
- Multiplication would not be correct using 2's complement values.
- Cannot use the same multiplication operation in signed and unsigned values. Focus on unsigned here.
- For more information: https://bohr.wlu.ca/cp216/docs/QRC0001_UAL.pdf

Unsigned multiplications

Operation	Assembler	Action
Multiply	`MUL{S} Rd, Rm, Rs`	Rd := (Rm * Rs)[31:0]
and accumulate	`MLA{S} Rd, Rm, Rs, Rn`	Rd := (Rn + (Rm * Rs))[31:0]
and subtract	`MLS Rd, Rm, Rs, Rn`	Rd := (Rn – (Rm * Rs))[31:0]
unsigned long	`UMULL{S} RdLo, RdHi, Rm, Rs`	RdHi,RdLo := unsigned(Rm * Rs)
unsigned accumulate long	`UMLAL{S} RdLo, RdHi, Rm, Rs`	RdHi,RdLo := unsigned(RdHi,RdLo + Rm * Rs)

- Cannot use the same register to specify both `Rd` and `Rm`.
- MUL, MLA and MLS have the multiplication results truncated to the lower-order 32 bits.
- UMULL and UMLAL store the full 64-bit result into two registers `RdLo` and `RdHi`.